

Computer graphics Lab3

Q 1. Implement DDA line drawing algorithm make sure the algorithm should work for all slope cases print the intermediate pixel value in all those cases

```
#include<GL/glut.h>
#include<bits/stdc++.h>
#include<vector>
using namespace std;
float x1=50 ,y_1=50 ;
float x2=100 ,y2=100;

void MYInit()
{
    glClearColor(0 , 0 , 0 ,0);
    glColor3f(1, 0 , 0);
    glPointSize(4.0f);

    glMatrixMode(GL_PROJECTION);
    gluOrtho2D(0 , 500, 0 , 500);
}

void ddaline(float x1 , float y_1 , float x2 , float y2)
{
    float dx=x2-x1;
    float dy=y2-y_1;

    int step;
    if(abs(dx)>abs(dy))
    {
        step=abs(dx);
    }
    else{
        step=abs(dy);
    }

    float xinc=dx/step;
    float yinc=dy/step;
    float x=x1;
    float y=y_1;

    glBegin(GL_POINTS);

    for (int i = 0; i <= step; i++)
    {
        cout << i << "\t" << x << "\t" << y
```

```

        << "\t(" << round(x) << ", " << round(y) << ")" << endl;

        glVertex2i(round(x), round(y));

        x=x+xinc;
        y=y+yinc;

    }
    glEnd();

}

void Draw()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glLineWidth(3.0f);
    ddaline(x1, y_1 ,x2, y2);

    glFlush();

}
int main(int c , char *v[])
{

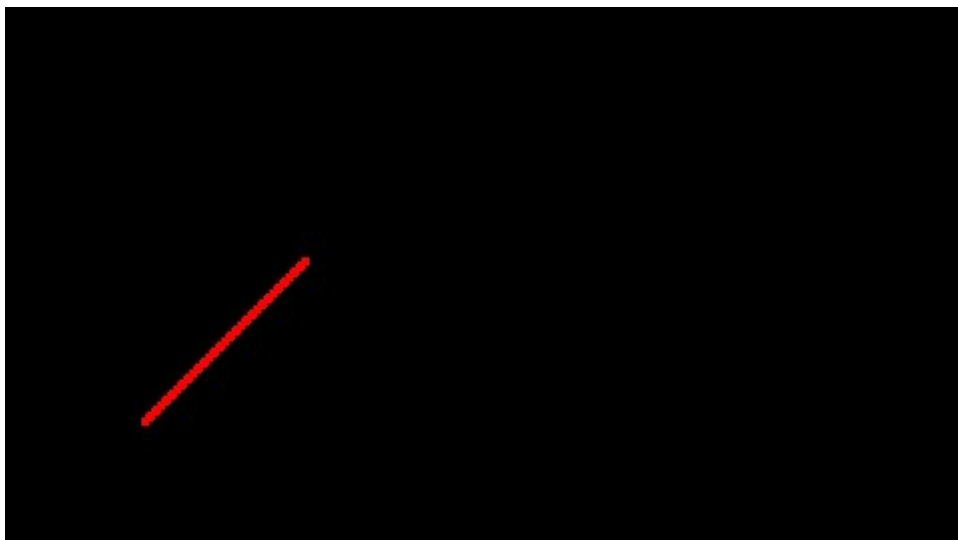
    glutInit(&c , v);

    glutInitDisplayMode(GLUT_RGB |GLUT_SINGLE);
    glutInitWindowSize(800 , 800);
    glutInitWindowPosition(200,200);
    glutCreateWindow("this is second opengle program");


    MYInit();
    glutDisplayFunc(Draw);
    glutMainLoop();
    return 0;

}

```



Q 2. Impliment Bresenham's line drawing algorithm sure all slop cases

```
#include <GL/glut.h>
#include <cmath>
using namespace std;

int x1 = 200, y_1 = 200;
int x2 = 1000, y2 = 100;

void MYInit()
{
    glClearColor(0, 0, 0, 0);
    glColor3f(1, 0, 0);
    glPointSize(4.0);

    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0, 500, 0, 500);
}

void bresenhamLine(int x1, int y_1, int x2, int y2)
{
    int dx = abs(x2 - x1);
    int dy = abs(y2 - y_1);

    int sx = (x1 < x2) ? 1 : -1;
    int sy = (y_1 < y2) ? 1 : -1;

    int p = dx - dy;

    glBegin(GL_POINTS);
    while (true)
    {
        glVertex2i(x1, y_1);

        if (x1 == x2 && y_1 == y2)
            break;

        int p2 = 2 * p;

        if (p2 > -dy)
        {
            p -= dy;
            x1 += sx;
        }

        if (p2 < dx)
        {
            p += dx;
            y_1 += sy;
        }
    }
}
```

```

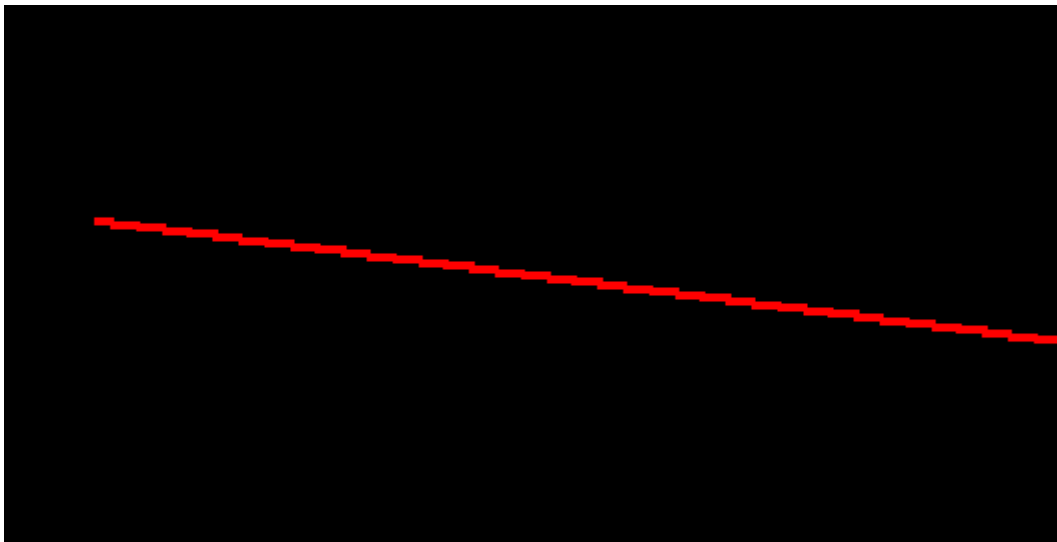
    }
}
glEnd();
}

void Draw()
{
    glClear(GL_COLOR_BUFFER_BIT);
    bresenhamLine(x1, y1, x2, y2);
    glFlush();
}

int main(int argc, char **argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_RGB | GLUT_SINGLE);
    glutInitWindowSize(800, 800);
    glutInitWindowPosition(200, 200);
    glutCreateWindow("Simple Bresenham Line");

    MYInit();
    glutDisplayFunc(Draw);
    glutMainLoop();
    return 0;
}

```



Q.3 IMPLEMENT ANY OF THE LINE DRAWING ALGORITHM TO DRAW FOLLOWING KINDS OF THE LINE

A. DOTTED LINE

```
#include<GL/glut.h>
#include<bits/stdc++.h>
#include<vector>
using namespace std;
float x1=100 ,y_1=100 ;
float x2=300 ,y2=300;
```

```
void MYInit()
{
    glClearColor(0 , 0 , 0 ,0);
    glColor3f(1, 1 , 1);
    glPointSize(4.0f);

    glMatrixMode(GL_PROJECTION);
    gluOrtho2D(0 , 500, 0 , 500);
}
```

```
void ddaline(float x1 , float y_1 , float x2 , float y2)
{
    float dx=x2-x1;
    float dy=y2-y_1;

    int step;
    if(abs(dx)>abs(dy))
    {
        step=abs(dx);
    }
    else{
```

```

        step=abs(dy);

    }

    float xinc=dx/step;
    float yinc=dy/step;
    float x=x1;
    float y=y_1;

    glBegin(GL_POINTS);

    for (int i = 0; i <= step; i++)
    {
        if(i%2==0)
            glVertex2i(round(x), round(y));

        x=x+xinc;
        y=y+yinc;
    }
    glEnd();
}

void Draw()
{

    glClear(GL_COLOR_BUFFER_BIT);
    glLineWidth(1);
    ddaline(x1, y_1 ,x2, y2);

    glFlush();
}

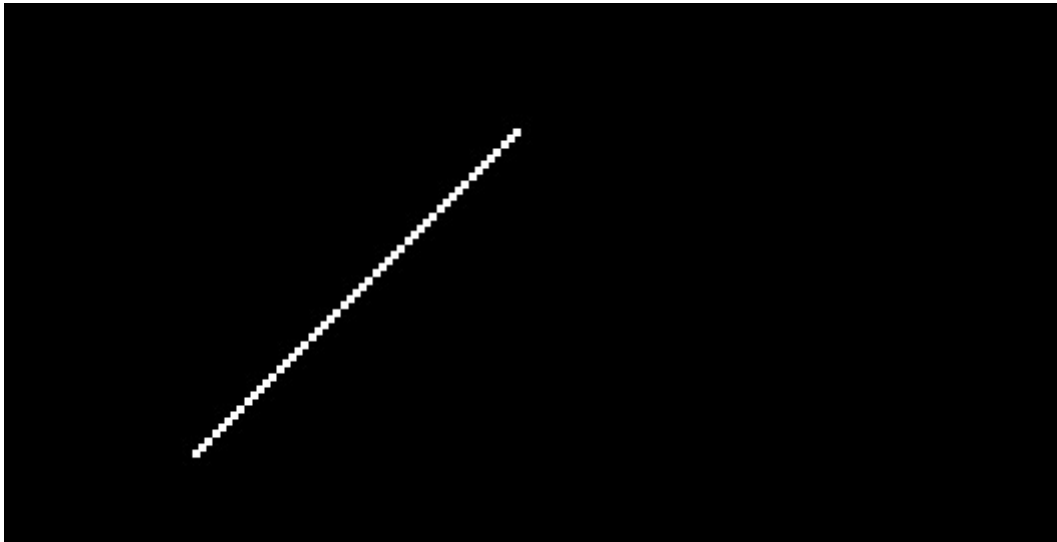
int main(int c , char *v[])
{

    glutInit(&c , v);

    glutInitDisplayMode(GLUT_RGB |GLUT_SINGLE);
    glutInitWindowSize(800 , 800);
    glutInitWindowPosition(200,200);
    glutCreateWindow("this is second opengle program");

    MYInit();
    glutDisplayFunc(Draw);
    glutMainLoop();
    return 0;
}

```



B.DASHED DOTTED LINE

```
#include <GL/glut.h>
#include <cmath>
using namespace std;

float x1 = 50, y_1 = 50;
float x2 = 200, y2 = 100;

void MYInit()
{
    glClearColor(0, 0, 0, 0);
    glColor3f(1, 0, 0);
    glPointSize(5.0f);

    glEnable(GL_POINT_SMOOTH);

    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0, 500, 0, 500);
}

void ddaDotDashLine(float x1, float y_1, float x2, float y2)
{
    float dx = x2 - x1;
    float dy = y2 - y_1;

    int step = (abs(dx) > abs(dy)) ? abs(dx) : abs(dy);

    float xinc = dx / step;
    float yinc = dy / step;
```

```
float x = x1;
float y = y_1;
```

```
int pattern[] = {1, 1, 5, 1};
int patternLength = 4;
int patternIndex = 0;
int current = 0;
```

```
glBegin(GL_POINTS);
for (int i = 0; i <= step; i++)
{
    if (patternIndex % 2 == 0)
        glVertex2i(round(x), round(y));

    x += xinc;
    y += yinc;

    current++;
    if (current >= pattern[patternIndex])
    {
        patternIndex = (patternIndex + 1) % patternLength;
        current = 0;
    }
}
glEnd();
```

```
}
```

```
void Draw()
{
    glClear(GL_COLOR_BUFFER_BIT);
    ddaDotDashLine(x1, y_1, x2, y2);
    glFlush();
}
```

```
int main(int argc, char **argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_RGB | GLUT_SINGLE);
    glutInitWindowSize(800, 800);
    glutInitWindowPosition(200, 200);
    glutCreateWindow("Dot-Dash Line - DDA Algorithm");

    MYInit();
    glutDisplayFunc(Draw);
    glutMainLoop();
    return 0;
}
```



Q 4. USE ANY LINE DRAWING ALGORITHM TO PRINT “HELLO”

```
#include <GL/glut.h>
#include <cmath>
using namespace std;

void ddaLine(float x1, float y1, float x2, float y2)
{
    float dx = x2 - x1;
    float dy = y2 - y1;
    int steps = (abs(dx) > abs(dy)) ? abs(dx) : abs(dy);

    float xInc = dx / steps;
    float yInc = dy / steps;

    float x = x1, y = y1;

    glBegin(GL_POINTS);
    for (int i = 0; i <= steps; i++)
    {
        glVertex2i(round(x), round(y));
        x += xInc;
        y += yInc;
    }
    glEnd();
}

void drawHello()
{
    glClear(GL_COLOR_BUFFER_BIT);
```

```
ddaLine(50, 200, 50, 100);
ddaLine(80, 200, 80, 100);
ddaLine(50, 150, 80, 150);
```

```
ddaLine(120, 200, 120, 100);
ddaLine(120, 200, 160, 200);
ddaLine(120, 150, 155, 150);
ddaLine(120, 100, 160, 100);
```

```
ddaLine(180, 200, 180, 100);
ddaLine(180, 100, 220, 100);
```

```
ddaLine(240, 200, 240, 100);
ddaLine(240, 100, 280, 100);
```

```
ddaLine(300, 200, 340, 200);
ddaLine(300, 100, 340, 100);
ddaLine(300, 100, 300, 200);
ddaLine(340, 100, 340, 200);
```

```
    glFlush();
}
```

```
void MYInit()
{
    glClearColor(0, 0, 0, 0);
    glColor3f(1, 1, 0);
    glPointSize(3.0f);

    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0, 500, 0, 500);
}
```

```
int main(int argc, char **argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_RGB | GLUT_SINGLE);
    glutInitWindowSize(600, 400);
    glutInitWindowPosition(200, 200);
    glutCreateWindow("Draw HELLO - DDA Line Drawing");

    MYInit();
    glutDisplayFunc(drawHello);
    glutMainLoop();
    return 0;
}
```

HELLO